Logistic regression

SPRING 2023

ZORAN TIGANJ

Logistic Regression

- Logistic Regression (also called Logit Regression) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?).
  - If the estimated probability is greater than 50% , then the model predicts that the instance belongs to that class (called the positive class, labeled “1”),
  - otherwise it predicts that it does not (i.e., it belongs to the negative class, labeled “0”).
  - This makes it a **binary classifier**.

Estimating Probabilities

- Just like a Linear Regression model, a Logistic Regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the Linear Regression model does, it outputs the logistic of this result

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\left(\mathbf{x}^{\mathsf{T}}\boldsymbol{\theta}\right)$$

- The logistic—noted σ(·)—is a sigmoid function (i.e., S-shaped) that outputs a number between 0 and 1.

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

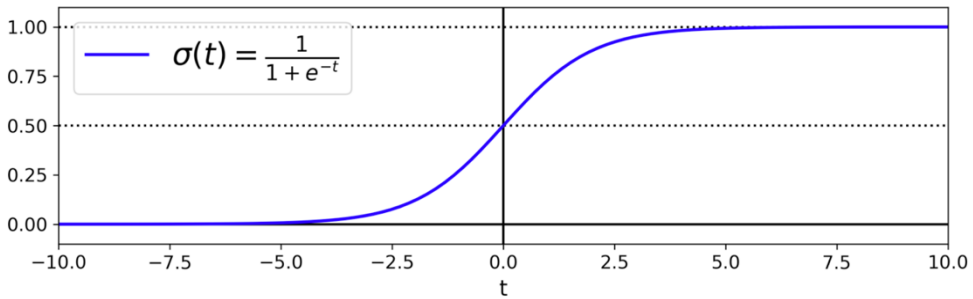


Figure 4-21. Logistic function

Estimating Probabilities

- Once the Logistic Regression model has estimated the probability $\hat{p} = h_{\Theta}(x)$ that an instance x belongs to the positive class, it can make its prediction ŷ easily

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

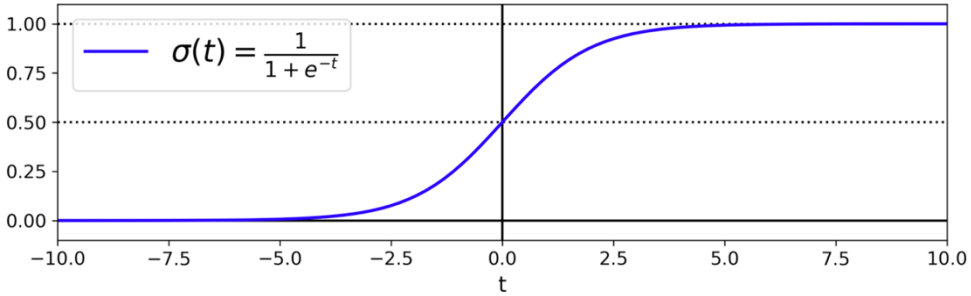


Figure 4-21. Logistic function

Estimating Probabilities

- Once the Logistic Regression model has estimated the probability $\hat{p} = h_\Theta(x)$ that an instance x belongs to the positive class, it can make its prediction ŷ easily

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

- Once the Logistic Regression model has estimated the probability $\hat{p} = h_\Theta(x)$ that an instance x belongs to the positive class, it can make its prediction ŷ easily

Notice that $\sigma(t) < 0.5$ when $t < 0$, and $\sigma(t) \geq 0.5$ when $t \geq 0$, so a Logistic Regression model predicts 1 if $\mathbf{x}^\intercal \boldsymbol{\theta}$ is positive and 0 if it is negative.

Estimating Probabilities

- The score $t = \boldsymbol{x}^T \boldsymbol{\Theta}$ is often called the logit.
- Sidenote: the name comes from the fact that the logit function, defined as
  - logit(p) = log(p / (1 – p))

  is the inverse of the logistic function.

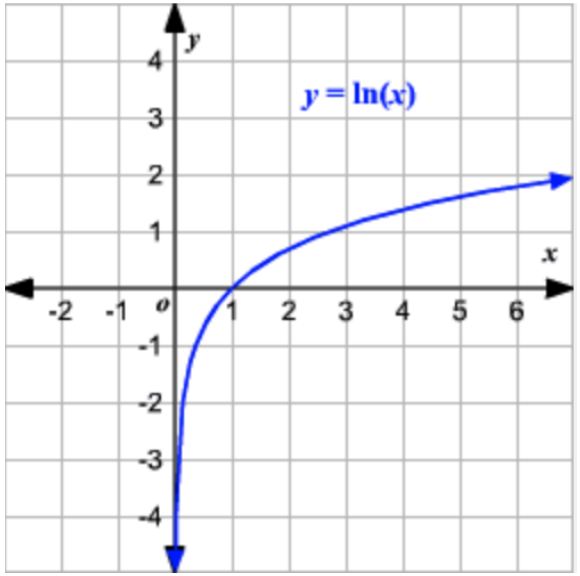
  If you compute the logit of the estimated probability p, you will find that the result is t.

  The logit is also called the log-odds, since it is the log of the ratio between the estimated probability for the positive class and the estimated probability for the negative class.

Training and Cost Function

- The objective of training is to set the parameter vector **θ** so that the model estimates high probabilities for positive instances (y = 1) and low probabilities for negative instances (y = 0).

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1-\hat{p}) & \text{if } y = 0 \end{cases}$$



- This cost function makes sense because –log($\hat{p}$) grows very large when $\hat{p}$ approaches 0, so the cost will be large if the model estimates a probability close to 0 for a positive instance, and it will also be very large if the model estimates a probability close to 1 for a negative instance.
- On the other hand, –log($\hat{p}$) is close to 0 when t is close to 1, so the cost will be close to 0 if the estimated probability is close to 0 for a negative instance or close to 1 for a positive instance, which is precisely what we want.

Training and Cost Function

- The cost function over the whole training set is the average cost over all training instances. It can be written in a single expression called the log loss

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log\left(\hat{p}^{(i)}\right)+\left(1-y^{(i)}\right)log\left(1-\hat{p}^{(i)}\right)\right]$$

Maximum likelihood perspective for logistic regression

- Let y be the true label (0 or 1) and $\hat{p}$ be the predicted probability of y=1. The likelihood L of observing y given $\hat{p}$ is:
- If we have just one observation: $L(\Theta|\boldsymbol{X},\boldsymbol{y}) = \hat{p}^{y}(1-\hat{p})^{1-y}$
  - if y=1, $L(y|\ \hat{p})= \hat{p}$
  - if y=0, $L(y|\ \hat{p})=1-\hat{p}$
- We usually deal with multiple data points $(y_1,p_1),(y_2,p_2),\ldots,(y_N,p_N)$. The likelihood of observing all the data points is the product of the individual likelihoods:
- $L(\Theta|\boldsymbol{X},\boldsymbol{y}) = \prod_{i=1}^{N}\hat{p}_i^{\,y_i}(1-\hat{p}_i)^{1-y_i}$
- To simplify this expression and turn it into a minimization problem, we take the negative log-likelihood:
- $-\log(L) = -\sum_{i=1}^{N} y_i\log(\hat{p}_i) \quad + (1-y_i)log(1-\hat{p}_i)$
- This is exactly the cross-entropy loss function for binary classification.

In intuitive terms, the likelihood function quantifies how well a particular model explains the observed data.

Maximum likelihood perspective for linear regression

- In the case of linear regression, the model predicts a continuous output $\hat{\boldsymbol{y}}$ based on input features X and model parameters $\boldsymbol{\theta}$. Typically, we assume that the output $\boldsymbol{y}$ is a linear combination of the input features plus some random error ϵ that follows a Normal (Gaussian) distribution with mean 0 and variance σ^2:
- $\boldsymbol{y} = \hat{\boldsymbol{y}} + \epsilon = \boldsymbol{X}^T\boldsymbol{\theta} + \epsilon$
- Given this model, the likelihood of observing a particular data point (xi,yi) is given by the probability density function of a Gaussian distribution:

$$L(y_i|\hat{y}_i, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right)$$

Maximum likelihood perspective for linear regression

- For *N* data points, the likelihood of observing the entire dataset

$$L = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right)$$

- Taking the negative log-likelihood, we get:

$$-\log(L) = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- The Mean Squared Error (MSE) loss function is directly analogous to one of the terms in the negative log-likelihood:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Back to Logistic Regression: Training and Cost Function

- The cost function over the whole training set is the average cost over all training instances. It can be written in a single expression called the log loss

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log\left(\hat{p}^{(i)}\right)+\left(1-y^{(i)}\right)log\left(1-\hat{p}^{(i)}\right)\right]$$

- The bad news is that there is no known closed-form equation to compute the value of **θ** that minimizes this cost function (there is no equivalent of the Normal Equation).
- The good news is that this cost function is convex, so Gradient Descent (or any other optimization algorithm) is guaranteed to find the global minimum (if the learning rate is not too large and you wait long enough).

Training and Cost Function

- The partial derivatives of the cost function with regard to the jth model parameter θ_j are given by

$$\frac{\partial}{\partial \theta_j} \mathrm{J}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \left(\sigma\left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)}\right) - y^{(i)} \right) x_j^{(i)}$$

- Once you have the gradient vector containing all the partial derivatives, you can use it in the Batch Gradient Descent algorithm.

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\theta)$$

Decision Boundaries

- Let's use the iris dataset to illustrate Logistic Regression.
- This is a famous dataset that contains the sepal and petal length and width of 150 iris flowers of three different species:
  - Iris setosa
  - Iris versicolor
  - Iris virginica
- Let's try to build a classifier to detect the Iris virginica type based only on the petal width feature.

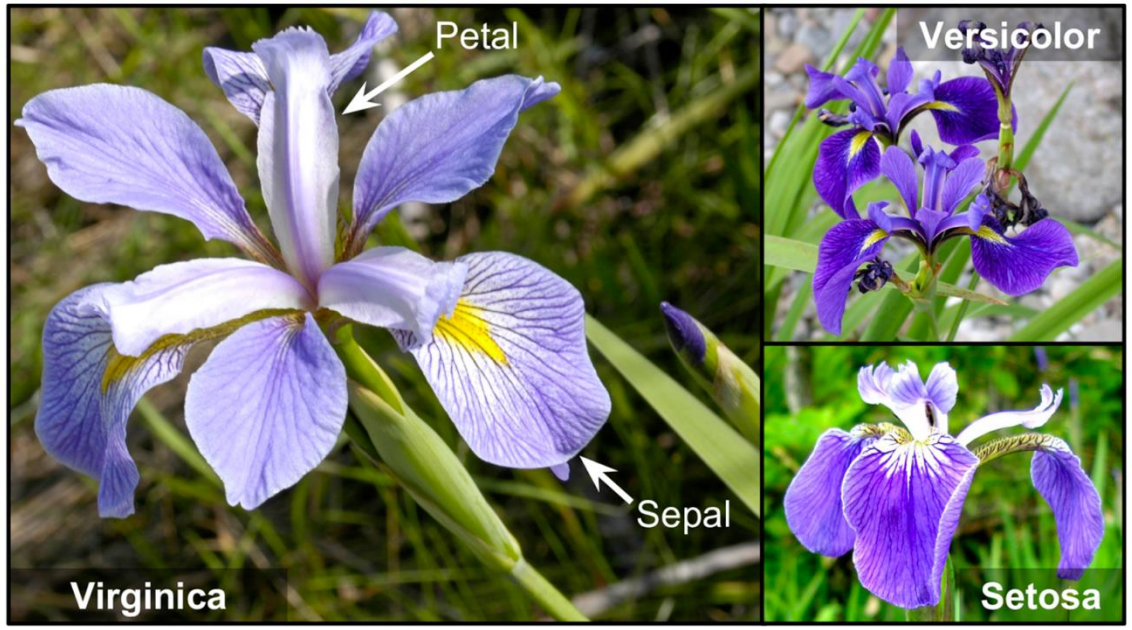


Figure 4-22. Flowers of three iris plant species[14]

Decision Boundaries

First let's load the data:

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> list(iris.keys())
['data', 'target', 'target_names', 'DESCR', 'feature_names', 'filename']
>>> X = iris["data"][:, 3:]  # petal width
>>> y = (iris["target"] == 2).astype(np.int)  # 1 if Iris virginica, else 0
```

Now let's train a Logistic Regression model:

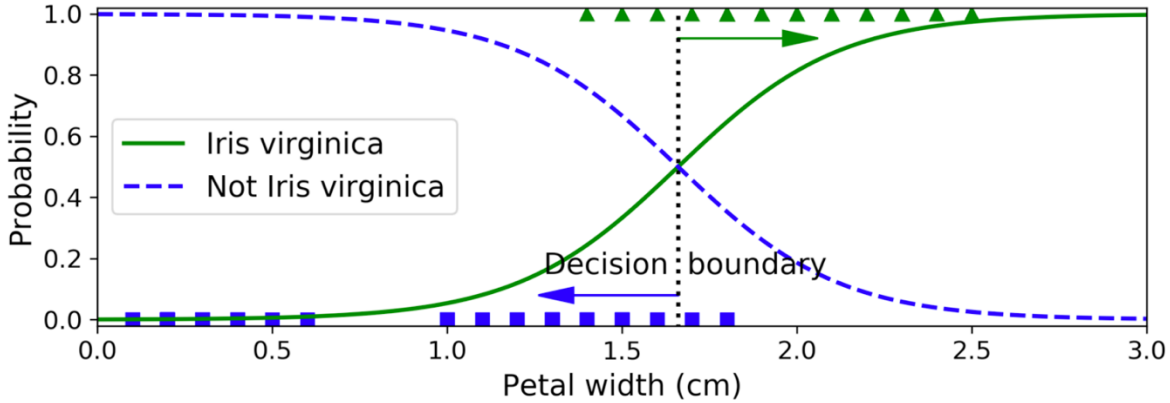
```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression()
log_reg.fit(X, y)
```

Decision Boundaries

- Let's look at the model's estimated probabilities for flowers with petal widths varying from 0 cm to 3 cm

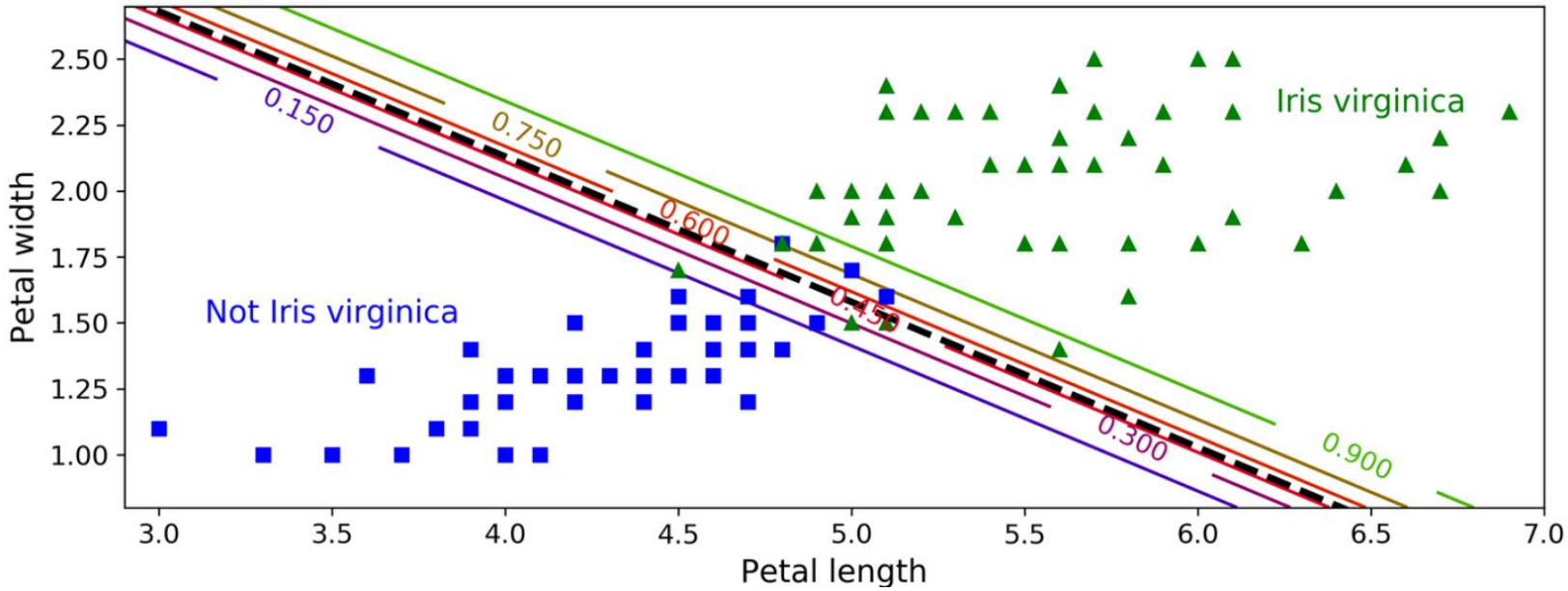
```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:, 1], "g-", label="Iris virginica")
plt.plot(X_new, y_proba[:, 0], "b--", label="Not Iris virginica")
# + more Matplotlib code to make the image look pretty
```




```
>>> log_reg.predict([[1.7], [1.5]])
array([1, 0])
```

Decision Boundaries

- Two features:



Regularization

- Just like the other linear models, Logistic Regression models can be regularized using ℓ_1 or ℓ_2 penalties.

Next time

- Softmax regression (Chapter 4)